

ANDROID ARCHITECTURE SERIES

ViewModel에 흠어진 화면 흐름을 상태 머신(FSM)으로 묶는 Afsm

Boolean 지옥을 넘어 순수 Kotlin으로 구축하는 예측 가능하고 안전한 화면 아키텍처

PRESENTER & TEAM

Kez Lab • Android Architecture
Team

SESSION & DATE

Tech Deep Dive • 2026.08

OPEN SOURCE HUB

github.com/kez-lab/afsm

AGENDA

CONTENTS

ViewModel 한계

극복부터

Afsm 실전 적용까지의

4단계

01

Boolean 지옥과 한계

- 기능 추가에 따른 상태 조합 폭발
- 코루틴 비동기 분기 파편화
- 회귀 버그 유발 위험

02

Afsm 3대 핵심 모델

- State (Phase + Data 결합)
- Event (방금 발생한 사실)
- Command (Host 요청 작업)

03

협업 아키텍처

- 순수 머신과 ViewModel의 분업
- 단방향 데이터 흐름(UDF) 보장
- 부작용 없는 전이 함수 설계

04

테스트 & 도입 가이드

- Mock 없는 0ms 순수 단위 테스트
- Mermaid 다이어그램 자동 동기화
- 상태 머신 적용 판단 기준

PROBLEM STATEMENT

Boolean이 하나씩 추가될수록 화면의 흐름은 사라집니다

개별 변수는 맞아 보이지만, 한 화면에 모아 놓으면 절대 공존할 수 없는 무효 상태가 발생합니다.

01

상태 조합 폭발

isSaving && isSaved 같은 무효 상태가 동시에 참이 됨.

변수 N개 시 2^N 개 경우의 수를 개발자가 머리로 외워야 함

02

비동기 분기 파편화

중복 클릭 방지, 재시도, 늦게 도착한 응답 처리가 여러 함수로 분산.

코드 한 줄씩은 맞는데 전체 흐름 파악 곤란

03

높은 테스트 비용

단순 비즈니스 분기 하나 검증하려 해도 Repository Mock 필수.

TestDispatcher 및 코루틴 타이밍 의존

우리는 질문을 근본적으로 바꿔야 합니다

**“ 상태 값을 어떻게 잘 바꿀까? 가 아니라,
현재 단계에서 이 이벤트를 받아도 되는가? 를
코드에 그대로 드러내는 것입니다. ”**

Afsm (Android Finite State Machine)의 핵심 설계 철학

✓ 상호 배타적 Phase 정의 ✓ 허용된 Event만 처리 ✓ 외부 작업은 Command로 격리

Afsm의 3대 공개 타입: State, Event, Command

화면의 단계, 일어난 사실, 앞으로 할 외부 작업을 엄격히 분리합니다.

01

STATE (Phase + Data)

- **Phase:** Editing, Saving, Saved
단일 단계
- **Data:** Title, Error 등 비즈니스
데이터
- 상호 배타적 단계로 무효 조합 원천
차단

02

EVENT (방금 일어난 일)

- 사용자 액션 (SaveClicked,
TitleChanged)
- 비동기 결과 (SaveCompleted,
Failed)
- 머신은 순수 동기 함수로 다음 전이
결정

03

COMMAND (요청할 외부 작업)

- Repository 호출 등 부작용 작업
명세서
- 머신 내부에서 suspend 호출 금지
- 승인된 전이에서만 Host에 작업
요청

순수 머신과 ViewModel의 단방향 데이터 흐름

Afsm은 ViewModel을 대체하지 않고, 비즈니스 전이 규칙만 순수 Kotlin으로 가져옵니다.



KEY ADVANTAGES

Afsm 도입으로 얻는 3대 엔지니어링 가치

가장 빠르고 가벼운 단위 테스트부터 CI 레벨의 다이어그램 검증까지 지원합니다.

01

0 ms

Mock-Free 단위 테스트

Repository Mock이나
TestDispatcher 없이 원하는 State와
Event를 넣고 즉시 전이 검증

02

100%

코드 기반 Mermaid 생성

실행되는 머신 코드에서 상태
다이어그램이 자동 생성되며, CI에서
코드-설계 일치 검증

03

0 Error

부작용(Side-Effect) 격리

모든 외부 작업을 Command로
격리하여 중복 실행이나 상태 오염을
원천 차단

SUMMARY & NEXT STEPS

핵심 요약 및 질의응답 (Q&A)

복잡한 화면 흐름을 상태 머신으로 표준화하여 견고한 안드로이드 앱을 구축하세요.

01. 상호 배타적 Phase 정의

유효하지 않은 Boolean 상태 조합을 컴파일 타임에 원천 차단

02. 부작용(Side-Effect) Command 격리

전이 판단과 외부 작업을 분리하여 Mock 없는 0ms 단위 테스트 달성

03. Mermaid 다이어그램 자동 동기화

코드 기반 상태 다이어그램으로 설계와 구현의 100% 일치 자동 검증

OPEN FOR DISCUSSION

Q & A

궁금하신 점이나 아키텍처 도입 논의를 환영합니다.

GITHUB & DOCS HUB

github.com/kez-lab/afsm

CONTACT

kejprogramer0909@gmail.com